AutoOps

Self-Hosted CI/CD Platform MVP

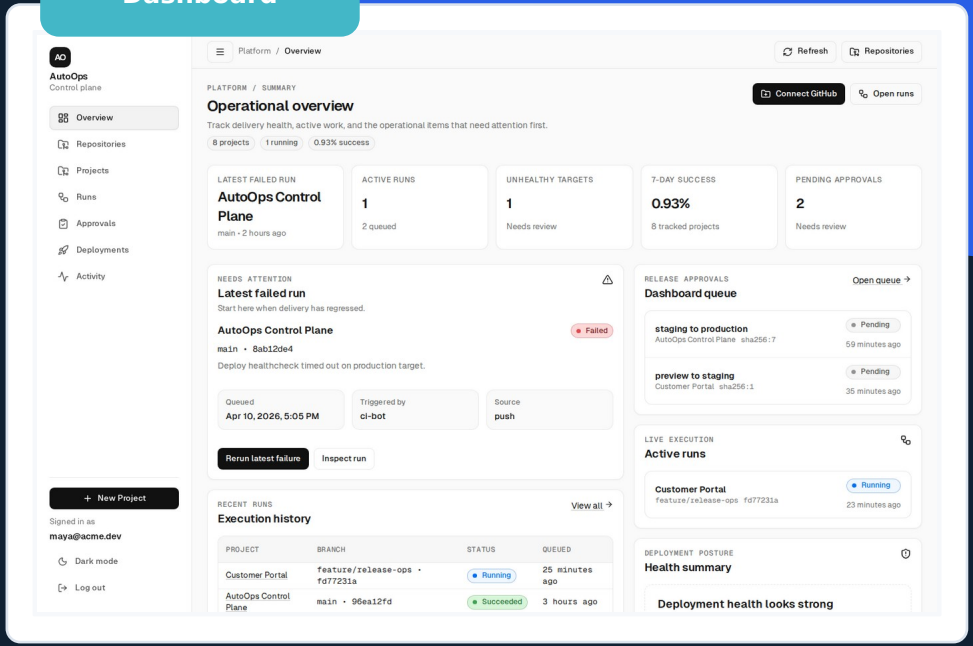
A control plane for GitHub-triggered pipelines, Docker-based execution, deployment health, approvals, and rollback-safe operations.

Express API | Worker | React Dashboard

Postgres | Docker | SSH + Managed Deploys

Anas Khan • Anshuman Tiwari • Bhumika Burhade • Chetali Aggrawal

Dashboard



What AutoOps is designed to solve

The problem AutoOps addresses and who it serves.

- Turns a GitHub push into a controlled, self-hosted CI/CD workflow.
- Centralizes queueing, execution, deployment state, and recovery.
- Strongest where operations hurt: stage visibility, approval gates, and rollback.
- Supports both custom pipelines and managed imports for web applications.

Core promise

Push code. Track every stage. Deploy safely. Recover quickly.

Primary users

Platform teams, operators, and self-hosted product teams that need more control.

What Phase 1 already includes

Six capabilities that cover the full CI/CD loop from intake to recovery.

GitHub Intake

App installations, webhook validation, deduplication, repository syncing, and onboarding metadata.

Pipeline Config

Per-repository .autoops/pipeline.yml with triggers, build settings, test commands, and deployment targets.

Execution Engine

Queued runs, ordered stages, Docker build and test execution, image push, and per-target deployment.

Operations UI

Overview, repositories, projects, runs, deployments, approvals, activity feeds, and rollback actions.

Recovery Controls

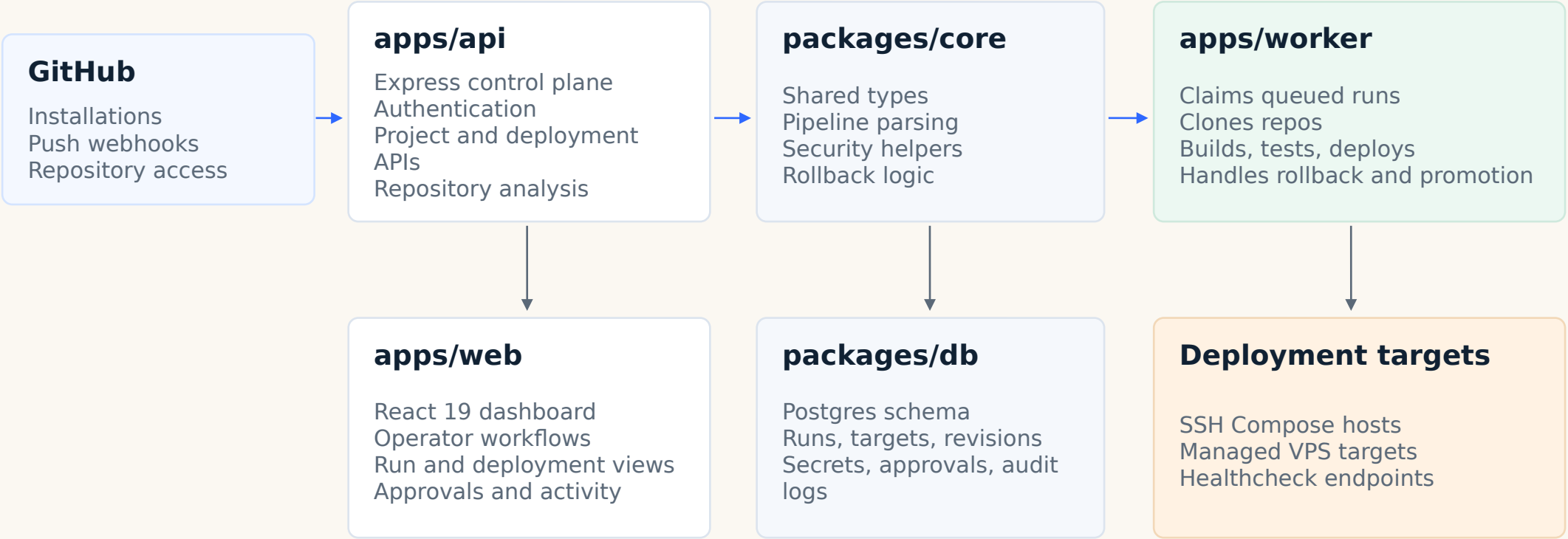
Deployment revisions, automatic rollback, manual rollback, and protected-target promotion approvals.

Managed Mode

Supported imports for standalone Next.js, React, CRA, and static HTML applications.

How the system is split up

Clean separation between control plane, execution, product UI, and shared platform logic.



From webhook to deployed revision

Every push follows this sequence from webhook to deployed revision.

Push event

GitHub emits a signed push webhook.

→

Validation

API verifies headers and deduplicates the delivery.

→

Queueing

Matching projects create queued runs with branch and SHA.

→

Prepare

Worker clones the repo and resolves the pipeline config.

→

Build + test

Docker image build and test commands run in sequence.

→

Deploy

Image push, target deploy, healthcheck, revision record.

Persistent state

Runs, stages, logs, delivery outcomes, targets, revisions, and approvals all land in Postgres for traceability.

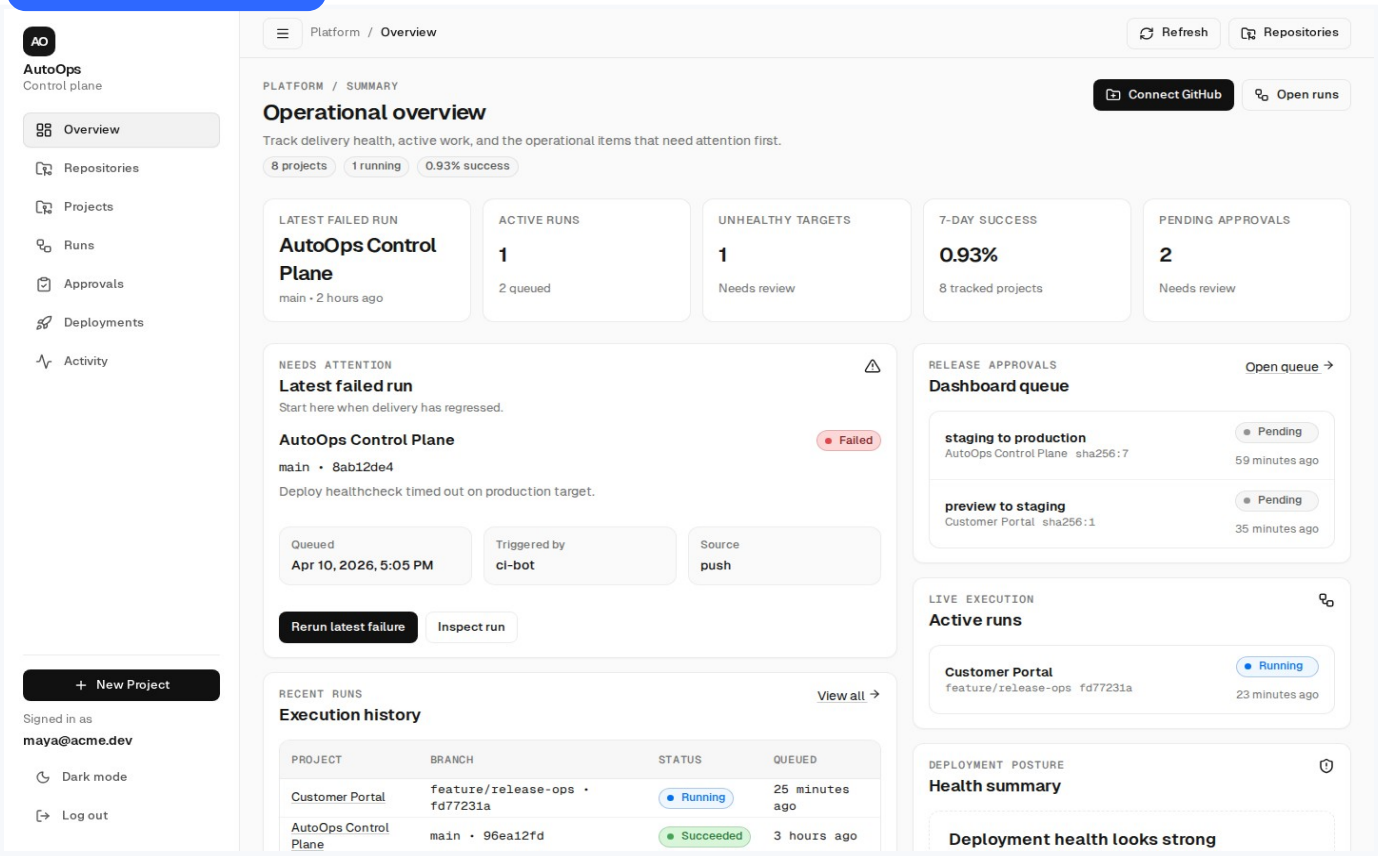
Operator result

The dashboard can immediately show what failed, where it failed, and what the next safe recovery action should be.

Overview page for the operator

The dashboard starts with the things that need attention first.

Overview



What it shows

Project count, running work, success rate, unhealthy targets, and pending approvals.

Why it matters

The page is tuned for triage: latest failure, active work, and fast rerun access are all surfaced immediately.

GitHub connection and repository discovery

This is where AutoOps links source repositories to managed or custom deployment paths.

GitHub integration

The UI brings together OAuth connection, installation sync, repository inventory, and import actions in one workflow.

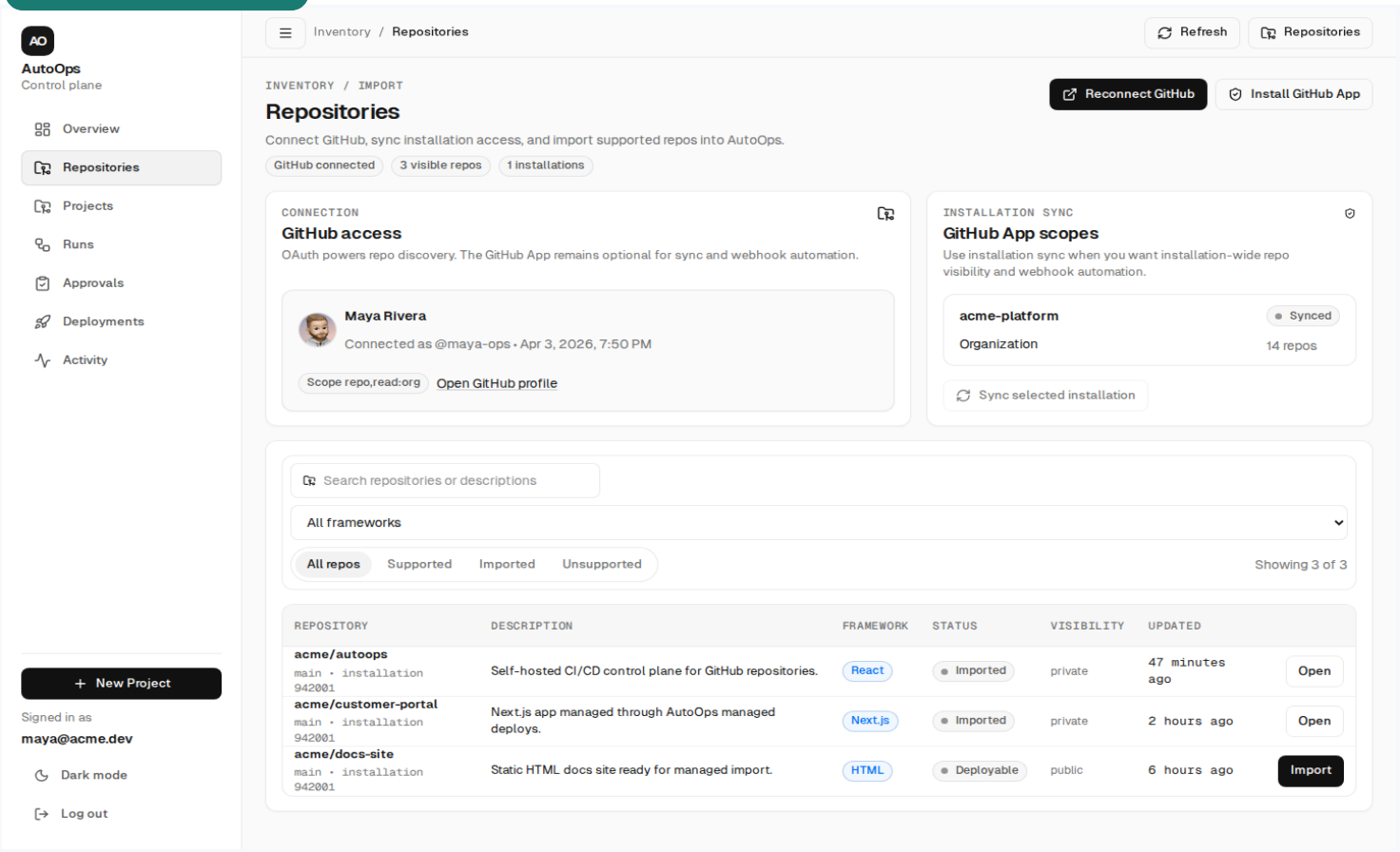
Managed import path

Supported repos can be analyzed and imported directly into managed deployments when they match v1 constraints.

Version 1 boundary

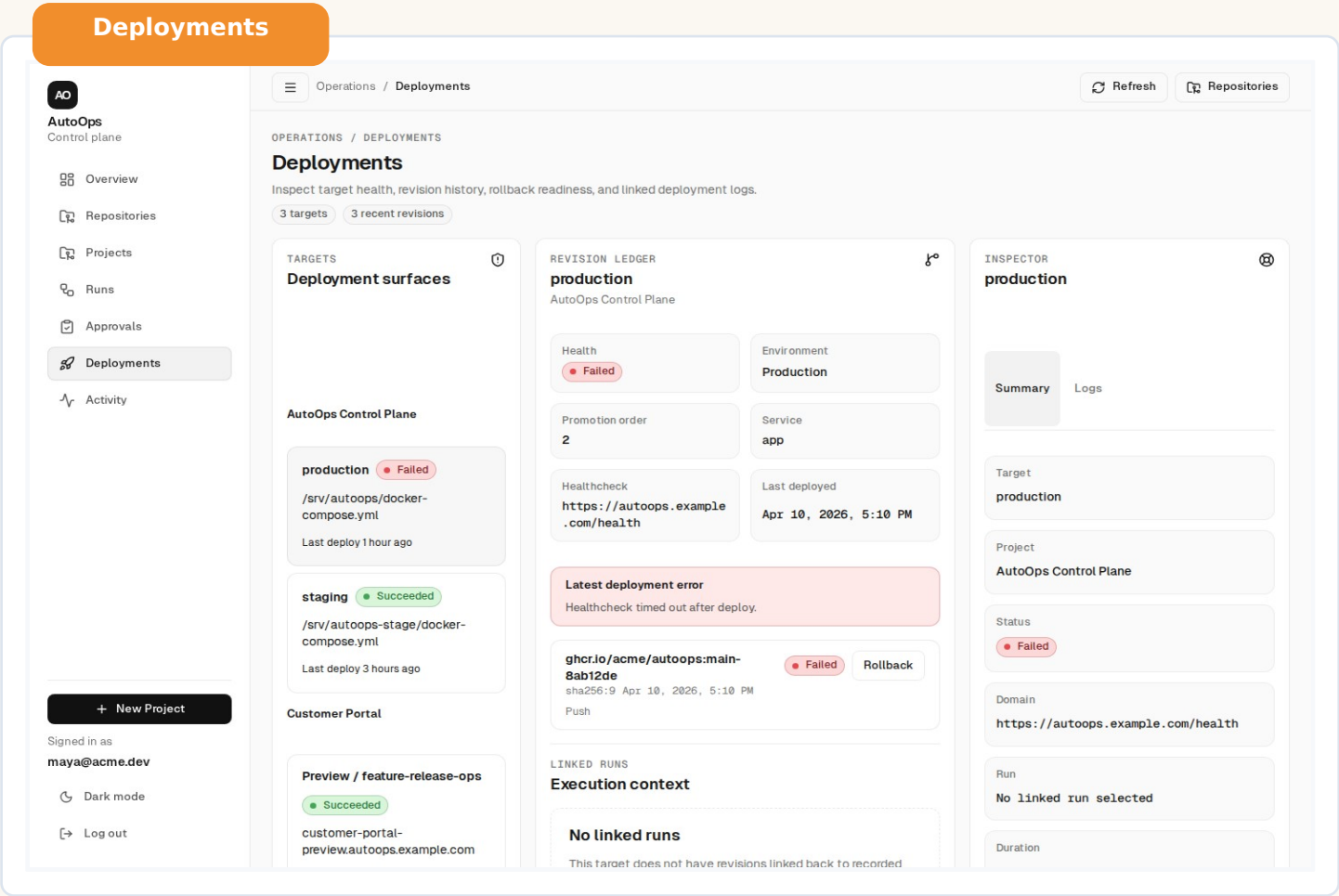
Managed mode targets standalone Next.js, React, CRA, and static HTML repositories. Monorepos stay on the custom-pipeline route.

Repositories



Deployment safety at a glance

The deployment view makes release safety concrete with revisions, approvals, and target-level status.



- Targets carry environment, protection status, promotion order, healthcheck URL, and deployment metadata.
- Successful releases become deployment revisions with image references and digests, which makes rollback explicit.
- If deployment health fails after a known-good revision exists, the worker can attempt automatic rollback.

Protected release approvals and traceability

Approvals are first-class workflow objects, not comments pasted into chat.

Approval model

Protected promotions can require explicit approval before deployment work is queued.

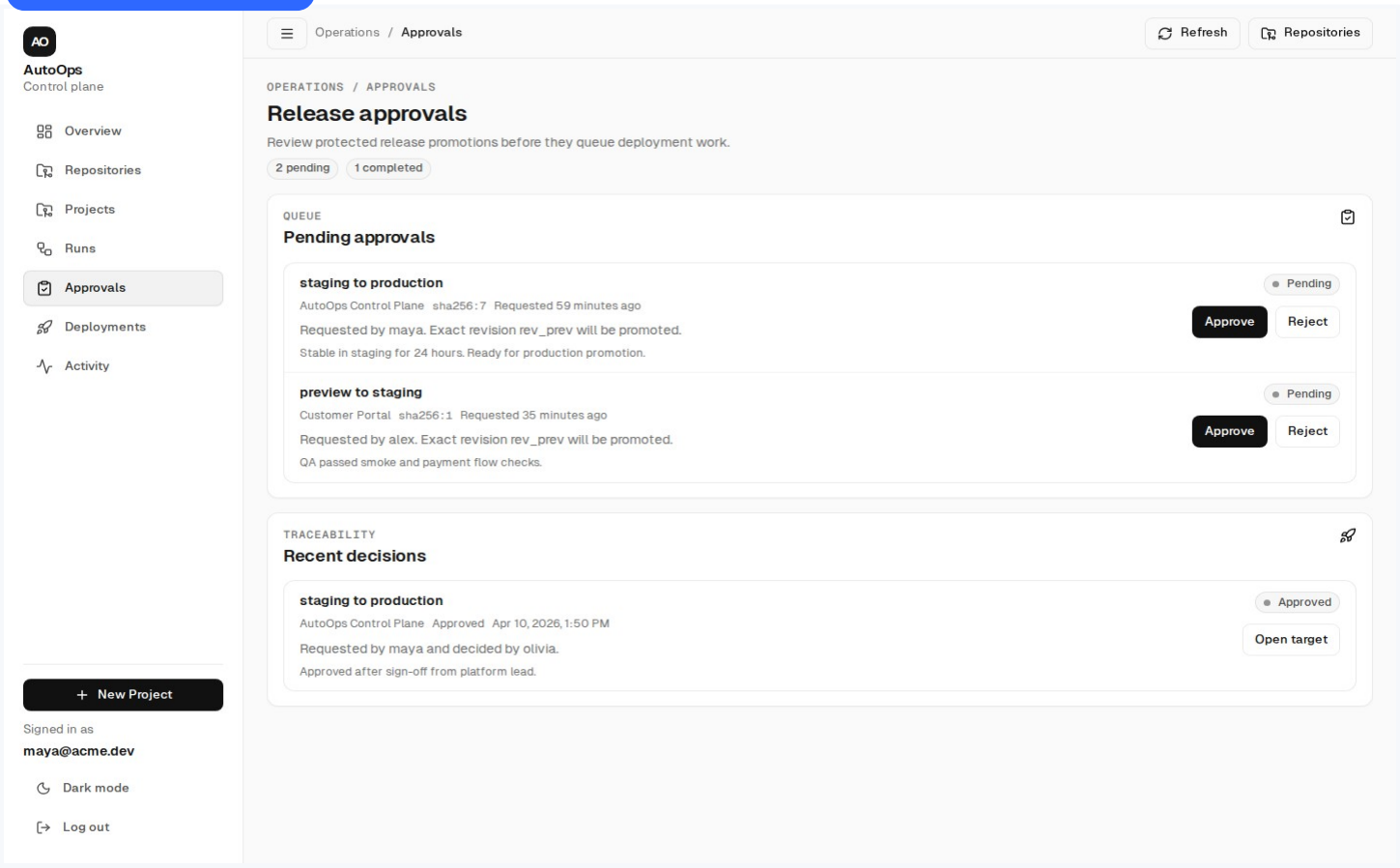
Stored context

Each record keeps requester, revision, source target, destination target, comments, and decision timestamps.

Why it matters

Operators get a durable change trail for protected releases instead of relying on memory or side-channel approvals.

Approvals



Test coverage and engineering discipline

Every package has tests, and the pipeline validates correctness before deployment.

17 test suites passing

API, worker, core, database, and web packages each have dedicated test files covering critical paths.

68 test cases

Webhook validation, pipeline parsing, deployment rollback, run streaming, and UI rendering all covered.

26 test files across 5 packages

Tests live alongside source in every package: apps/api, apps/web, apps/worker, packages/core, packages/db.

52 source modules

14,400+ lines of TypeScript across API routes, worker stages, React pages, shared logic, and database schema.

Monorepo with pnpm workspaces

Shared packages ensure type safety between API, worker, and dashboard without duplication.

Vitest + Testing Library

Fast unit and integration tests using modern tooling with DOM testing for the React dashboard.

What comes next in Phase 2

Planned extensions that build on the current foundation without rearchitecting.

Multi-environment promotion

Formalized staging → production chains with environment-specific configs and approval gates.

Kubernetes targets

Deploy to k8s clusters alongside existing Docker Compose and managed VPS targets.

Role-based access control

Team-scoped permissions for projects, approvals, and deployment targets.

Deeper observability

Build time trends, deploy frequency tracking, and integration with Grafana or Prometheus.

Broader framework support

Monorepo detection, Rust, Go, and Python pipelines beyond the current web-app focus.

Audit & compliance log

Immutable event log for every deployment decision, approval, and rollback action.

Live Demo

Webhook intake → pipeline run → deployment → healthcheck → rollback

Push to main | **Watch the build** | **Inspect the deploy**

Build with control. Deploy with confidence.

→ Full CI/CD loop: webhook intake, build, test, deploy, healthcheck, and rollback.

→ Operator-first dashboard with triage, approval gates, and recovery actions.

→ Clean monorepo architecture ready to extend with new deployment targets.

Live demo • github.com/autoops • Questions?